Projekt hurtowni danych Olist

Dokumentacja projektu zaliczeniowego

Hurtownie danych - laboratoria

Pole	Wartość
Zbiór danych	Olist Brazilian E-Commerce Public Dataset
Bazy danych	olist - warstwa staging; olist_dwh - hurtownia danych
Technologie	Python, Microsoft SQL Server 2022, Docker, SQL, Power BI
Autorzy	Marcel Żukowski Jakub Wiraszka
Data	2026

1. Cel projektu

Celem projektu było przygotowanie kompletnego przykładu hurtowni danych dla danych e-commerce. Projekt pokazuje pełny przepływ od danych źródłowych w plikach CSV, przez warstwę staging, proces ETL, model wymiarowy, walidacje oraz raportowanie SQL i Power BI.

Cel biznesowy rozwiązania to analiza sprzedaży, zamówień, dostaw, ocen klientów, kategorii produktów i sprzedawców na podstawie danych sklepu internetowego. Hurtownia pozwala odpowiadać m.in. na pytania: które kategorie generują największy przychód, które regiony mają najwięcej opóźnień, którzy sprzedawcy osiągają najlepsze wyniki oraz jaki jest wpływ opóźnień dostaw na oceny klientów.

2. Zakres wymagań i status realizacji

Wymaganie	Status	Gdzie zrealizowano / dowód
Model hurtowni danych	Wykonane	2_dwh_schema.sql, baza olist_dwh
Min. 1 tabela faktów	Wykonane	fact_orders, fact_order_items, fact_reviews, fact_payments
Min. 3 wymiary, w tym wymiar dat	Wykonane	dim_date, dim_customer, dim_product, dim_seller, dim_payment_type, dim_order_status, dim_geolocation
Proces ETL	Wykonane	main_staging.py, main_dwh.py, scripts/
Agregacje i transformacje	Wykonane	agg_monthly_sales, transformacje w main_dwh.py
Podstawowa obsługa delty danych	Wykonane	5_delta_demo.sql, load_status.json po stronie stagingu
3 przykładowe raporty	Wykonane	3_reports.sql oraz dashboardy Power BI
Dokumentacja i instrukcja uruchomienia	Wykonane	niniejszy dokument
Walidacje i logowanie	Wykonane jako element dodatkowy	4_validation_queries.sql, 6_etl_audit_schema.sql
SCD w wymiarach	Wykonane demonstracyjnie	7_scd_type2_product.sql - dim_product_scd2
Integracja z innym źródłem / dane live	Wykonane	fetch_nbp_brl_rate.py, 8_external_nbp_rates.sql, API NBP
Power BI / narzędzie BI	Wykonane	raport Power BI: Sprzedaż, Dostawy, Sprzedawcy/Kategorie
SSAS / SSIS	Nie wykonano	Zakres pominięty świadomie, aby nie komplikować działającego rozwiązania

3. Dane źródłowe

Projekt korzysta z publicznego zbioru Olist Brazilian E-Commerce Public Dataset. Dane są dostarczane jako pliki CSV i opisują zamówienia, klientów, sprzedawców, produkty, płatności, oceny oraz lokalizacje. W projekcie nie dołącza się dużych plików danych do paczki oddawanej na Moodle; zamiast tego w folderze data znajduje się opis, skąd pobrać dane i jakie pliki należy umieścić w projekcie.

Plik CSV	Opis
olist_customers_dataset.csv	klienci, miasto, stan, prefix kodu pocztowego
olist_orders_dataset.csv	zamówienia, statusy i daty realizacji
olist_order_items_dataset.csv	pozycje zamówień, produkty, sprzedawcy, ceny i fracht
olist_order_payments_dataset.csv	płatności, raty, typ płatności, wartość płatności
olist_order_reviews_dataset.csv	oceny i komentarze klientów
olist_products_dataset.csv	produkty, kategorie, wymiary i waga
olist_sellers_dataset.csv	sprzedawcy i lokalizacja
olist_geolocation_dataset.csv	dane geograficzne kodów pocztowych
product_category_name_translation.csv	tłumaczenia kategorii produktów z portugalskiego na angielski

4. Technologie i narzędzia

Obszar	Technologia / narzędzie	Rola w projekcie
Baza danych	Microsoft SQL Server 2022	przechowywanie warstwy staging i hurtowni danych
Konteneryzacja	Docker Compose	lokalne uruchomienie SQL Servera
ETL	Python	sterowanie tworzeniem baz, ładowaniem CSV i transformacjami
Połączenie z bazą	pymssql	łączenie Pythona z SQL Serverem
Raporty SQL	SQL Server / VS Code mssql	uruchamianie zapytań analitycznych i walidacyjnych
BI	Power BI Desktop	dashboardy sprzedaży, dostaw oraz kategorii/sprzedawców
Dane live	API NBP + requests	pobranie kursu BRL/PLN i przeliczenie wartości zamówień

5. Struktura projektu

```
project/
  data/
    DATA.md
    *.csv                        # dane Olist pobrane z Kaggle, niekoniecznie oddawane w ZIP
  scripts/
    database.py                  # konfiguracja i połączenia z SQL Serverem
    loader.py                    # ładowanie CSV do stagingu
    schema.py                    # uruchamianie skryptów SQL
  1_staging_tables.sql           # tabele staging
  2_dwh_schema.sql               # model hurtowni danych
  3_reports.sql                  # podstawowe raporty SQL
  4_validation_queries.sql       # walidacje danych
  5_delta_demo.sql               # demonstracja delty danych
  6_etl_audit_schema.sql         # logowanie ETL i wyniki walidacji
  7_scd_type2_product.sql        # demonstracja SCD Type 2
  8_external_nbp_rates.sql       # widok przeliczający BRL na PLN
  9_advanced_reports.sql         # raporty zaawansowane SQL
  fetch_nbp_brl_rate.py          # pobranie kursu BRL/PLN z API NBP
  docker-compose.yml             # SQL Server w Dockerze
  main_staging.py                # stworzenie i zasilenie stagingu
  main_dwh.py                    # stworzenie i zasilenie hurtowni danych
  requirements.txt               # zależności Python
  reports/                       # plik Power BI .pbix
  screenshots/                   # screeny wyników do dokumentacji
```

6. Instrukcja uruchomienia projektu

Poniższe kroki zakładają pracę na Windowsie, w terminalu PowerShell lub terminalu w Visual Studio Code. Komendy należy wykonywać z głównego folderu projektu, czyli z miejsca, gdzie znajduje się docker-compose.yml.

```
# 1. Uruchomienie SQL Servera w Dockerze
docker compose up -d

# 2. Sprawdzenie, czy kontener działa
docker ps

# 3. Utworzenie i aktywacja środowiska Python
python -m venv .venv
.\.venv\Scripts\activate

# 4. Instalacja zależności
pip install -r requirements.txt
pip install requests

# 5. Załadowanie warstwy staging
python main_staging.py --force
```

```
# 6. Utworzenie i zasilenie hurtowni danych
python main_dwh.py --etl --force

# 7. Pobranie kursu BRL/PLN z NBP
python fetch_nbp_brl_rate.py
```

Pełne ładowanie danych może trwać dłużej ze względu na rozmiar datasetu. W dokumentacji jako główny dowód poprawnego wykonania ETL przedstawiono wyniki walidacji po zakończonym ładowaniu, a nie pełne logi z terminala.

```
(.venv) PS C:\Users\Marcel\Desktop\MERITO\Semestr III\Hurtownie danych\wsb_dwh> docker ps
CONTAINER ID   IMAGE                                       COMMAND                  CREATED        STATUS                  PORTS                                          NAMES
81cc41306805   mcr.microsoft.com/mssql/server:2022-latest  "/opt/mssql/bin/laun…"   27 hours ago   Up 8 hours (unhealthy)  0.0.0.0:1433->1433/tcp, [::]:1433->1433/tcp    olist-mssql
```

Rysunek 1. Kontener SQL Server uruchomiony w Dockerze. Status healthcheck jest oznaczony jako unhealthy, ale kontener działa, port 1433 jest wystawiony, a dalsze zapytania SQL potwierdzają poprawne połączenie z bazą.

7. Architektura rozwiązania

Projekt został zbudowany w układzie warstwowym: pliki CSV są ładowane do bazy staging olist, następnie proces ETL przekształca dane do bazy olist_dwh, gdzie dane są przechowywane w modelu wymiarowym. Na końcu wykonywane są raporty SQL oraz dashboardy Power BI.

```
CSV z Kaggle
    -> baza staging: olist
        -> ETL Python + SQL
            -> hurtownia danych: olist_dwh
                -> walidacje SQL
                -> raporty SQL
                -> Power BI
                -> widok PLN z kursem NBP
```

8. Model hurtowni danych

Model hurtowni jest oparty na strukturze gwiazdy. Dane opisowe znajdują się w tabelach wymiarów, a dane mierzalne w tabelach faktów. Dodatkowo przygotowano tabelę agregującą agg_monthly_sales, która przyspiesza raportowanie miesięczne.

Typ	Tabela	Ziarnistość / rola
Wymiar	dim_date	jeden rekord na dzień, hierarchia czasu
Wymiar	dim_customer	jeden rekord na klienta, region klienta
Wymiar	dim_product	jeden rekord na produkt, kategoria, waga, wymiary
Wymiar	dim_seller	jeden rekord na sprzedawcę, region sprzedawcy
Wymiar	dim_payment_type	typ płatności i kategoria płatności
Wymiar	dim_order_status	status zamówienia i typ statusu
Wymiar	dim_geolocation	prefix kodu pocztowego, miasto, stan, współrzędne
Fakt	fact_order_items	jeden rekord na pozycję zamówienia
Fakt	fact_orders	jeden rekord na zamówienie
Fakt	fact_reviews	jeden rekord na ocenę
Fakt	fact_payments	jeden rekord na płatność / ratę
Agregacja	agg_monthly_sales	sprzedaż miesięczna wg kategorii i sprzedawcy

Przykład definicji wymiaru dat i tabeli agregującej:

```
CREATE TABLE dim_date (
    date_key INT PRIMARY KEY,
    calendar_date DATE NOT NULL UNIQUE,
```

```
    month_of_year INT NOT NULL,
    quarter INT NOT NULL,
    year INT NOT NULL,
    is_weekend BIT NOT NULL
);

CREATE TABLE agg_monthly_sales (
    year INT NOT NULL,
    month INT NOT NULL,
    category_name_en VARCHAR(200) NOT NULL,
    seller_key INT NOT NULL,
    total_orders INT,
    total_items INT,
    total_revenue DECIMAL(15, 2),
    avg_review_score DECIMAL(3, 2),
    orders_delayed INT,
    PRIMARY KEY (year, month, category_name_en, seller_key)
);
```

Wymiar dat wykorzystuje klucz date_key w formacie YYYYMMDD, co ułatwia sortowanie i łączenie z tabelami faktów. Tabela agg_monthly_sales agreguje dane miesięczne, dzięki czemu raporty nie muszą za każdym razem liczyć wyników z najniższego poziomu szczegółowości.

9. Proces ETL

Proces ETL składa się z dwóch głównych etapów. Pierwszy etap ładuje surowe pliki CSV do bazy staging olist. Drugi etap tworzy hurtownię olist_dwh i przekształca dane do wymiarów, faktów oraz agregacji.

Etap	Plik	Opis
Staging	main_staging.py	tworzy bazę olist, tabele staging i ładuje dane CSV
DWH	main_dwh.py	tworzy bazę olist_dwh, wymiary, fakty i agregacje
Schemat staging	1_staging_tables.sql	definicje tabel odpowiadających plikom źródłowym
Schemat DWH	2_dwh_schema.sql	definicje wymiarów, faktów, kluczy i indeksów
Loader	scripts/loader.py	mechanizm ładowania CSV do SQL Servera

Fragment main_staging.py odpowiedzialny za ładowanie danych CSV:

```
if not skip_data_load:
    data_loader = DataLoader(db_conn)
    if not data_loader.load_all_data(
        skip_existing=not force_recreate,
        reload_tables=reload_tables,
    ):
        logger.warning("Some CSV data failed to load")
```

W tym fragmencie skrypt tworzy obiekt DataLoader i uruchamia ładowanie wszystkich plików źródłowych. Parametr skip_existing pozwala pominąć dane, które były już wcześniej załadowane, co ułatwia ponowne uruchamianie projektu.

Przykład transformacji regionu klienta w wymiarze dim_customer:

```
INSERT INTO dim_customer (
    customer_id, customer_zip_code_prefix, customer_city,
    customer_state, region_name, customer_segment
)
SELECT DISTINCT
    c.customer_id,
    c.customer_zip_code_prefix,
    c.customer_city,
```

```
    c.customer_state,
    CASE c.customer_state
        WHEN 'SP' THEN 'Southeast'
        WHEN 'RJ' THEN 'Southeast'
        WHEN 'PR' THEN 'South'
        WHEN 'BA' THEN 'Northeast'
        ELSE 'Unknown'
    END AS region_name,
    'Regular'
FROM olist.dbo.olist_customers c;
```

Transformacja pokazuje, że hurtownia nie kopiuje danych bezpośrednio 1:1, ale wzbogaca je o atrybuty analityczne, np. region geograficzny wyprowadzony z kodu stanu Brazylii.

10. Walidacja danych

Po wykonaniu ETL uruchomiono zestaw zapytań walidacyjnych. Sprawdzono liczbę rekordów w tabelach, brakujące klucze wymiarów oraz zgodność przychodu między tabelą faktów i tabelą agregującą.

	table_name	row_count
1	dim_date	2192
2	dim_customer	99441
3	dim_seller	3095
4	dim_product	32951
5	dim_payment_type	5
6	dim_order_status	9
7	dim_geolocation	19015
8	fact_order_items	112650
9	fact_orders	99441
10	fact_reviews	97685
11	fact_payments	103886
12	agg_monthly_sales	24902

Rysunek 2. Liczba rekordów w tabelach DWH po wykonaniu ETL.

Tabela	Liczba rekordów
dim_date	2 192
dim_customer	99 441
dim_seller	3 095
dim_product	32 951
dim_payment_type	5

dim_order_status	9
dim_geolocation	19 015
fact_order_items	112 650
fact_orders	99 441
fact_reviews	97 685
fact_payments	103 886
agg_monthly_sales	24 902

	missing_customer_key	missing_product_key	missing_seller_key	missing_order_date_key
1	0	0	0	0

Rysunek 3. Walidacja brakujących kluczy w tabeli fact_order_items - wszystkie wartości wynoszą 0.

	revenue_from_fact_items	revenue_from_aggregate	difference
1	15843553.24	15843553.24	0.00

Rysunek 4. Spójność przychodu między fact_order_items i agg_monthly_sales - różnica wynosi 0.00.

Wyniki walidacji potwierdzają, że tabele faktów i wymiary zostały załadowane, w fact_order_items nie występują brakujące klucze do wymiarów, a suma przychodu z tabeli agregującej jest zgodna z sumą z tabeli faktów. Przychód z faktów i agregacji wyniósł 15 843 553.24, a różnica 0.00.

Fragment walidacji spójności przychodu:

```
SELECT
    (SELECT CAST(SUM(ISNULL(total_item_value, 0) + ISNULL(freight_value, 0)) AS DECIMAL(15, 2))
     FROM fact_order_items) AS revenue_from_fact_items,
    (SELECT CAST(SUM(total_revenue) AS DECIMAL(15, 2))
     FROM agg_monthly_sales) AS revenue_from_aggregate,
    ... AS difference;
```

11. Raporty SQL

W projekcie przygotowano zestaw raportów SQL. Minimalne wymaganie obejmowało trzy raporty, natomiast projekt zawiera raporty podstawowe i zaawansowane. Raporty służą do analizy sprzedaży, opóźnień, sprzedawców, metod płatności oraz kategorii produktów.

	year	month	category_name_en	total_orders	total_items	total_revenue	avg_order_value	delayed_orders
1	2016	9	health_beauty	1	3	143.46	143.46	1
2	2016	9	furniture_decor	1	2	136.23	136.23	0
3	2016	9	telephony	1	1	75.06	75.06	0
4	2016	10	furniture_decor	53	67	7052.28	133.06	0
5	2016	10	perfumery	30	33	6223.84	207.46	0
6	2016	10	health_beauty	44	48	5493.38	124.85	1
7	2016	10	toys	25	27	4986.08	199.44	0
8	2016	10	consoles_games	10	10	4089.42	408.94	0
9	2016	10	sports_leisure	19	19	3927.11	206.69	0

Rysunek 5. Raport 1 - miesięczna sprzedaż według kategorii produktu.

Raport miesięcznej sprzedaży pozwala analizować liczbę zamówień, liczbę pozycji, przychód, średnią wartość zamówienia i liczbę opóźnionych zamówień w rozbiciu na rok, miesiąc oraz kategorię produktu.

	region_name	total_orders	delayed_orders	delayed_percent	avg_days_to_delivery	total_revenue
1	Northeast	9394	1296	13.80	19.95	1880799.54
2	North	1851	176	9.51	22.54	410392.40
3	Center-West	5782	448	7.75	14.95	1023062.00
4	Southeast	68266	4933	7.23	10.69	10232323.50
5	South	14148	974	6.88	13.98	2296975.80

Rysunek 6. Raport 2 - opóźnienia dostaw według regionu klienta.

W raporcie region Northeast miał najwyższy udział opóźnionych zamówień, 13.80%. Region Southeast miał największą liczbę zamówień i największy przychód, ale procent opóźnień był niższy, 7.23%.

	revenue_rank	seller_id	seller_city	seller_state	region_name	total_orders	total_revenue	avg_review_score
1	1	4869f7a5dfa277a7dca6462dcf3b52b2	guariba	SP	Southeast	1132	249640.70	4.13
2	2	7c67e1448b00f6e969d365cea6b010ab	itaquaquecetuba	SP	Southeast	982	240518.07	3.35
3	3	4a3ca9315b744ce9f8e9374361493884	ibitinga	SP	Southeast	1806	237559.11	3.84
4	4	53243585a1d6dc2643021fd1853d8905	lauro de freitas	BA	Northeast	358	235856.68	4.08
5	5	fa1c13f2614d7b5c4749cbc52fecda94	sumare	SP	Southeast	585	204084.73	4.36
6	6	da8622b14eb17ae2831f4ac5b9dab84a	piracicaba	SP	Southeast	1314	186815.02	4.08
7	7	7e93a43ef30c4f03f38b393420bc753a	barueri	SP	Southeast	336	182754.05	4.22
8	8	1025f0e2d44d7041d6cf58b6550e0bfa	sao paulo	SP	Southeast	915	172999.23	3.87
9	9	7a67c85e85bb2ce8582c35f2203ad736	sao paulo	SP	Southeast	1160	162692.08	4.25

Rysunek 7. Raport 3 - ranking sprzedawców według przychodu.

Ranking sprzedawców wykorzystuje funkcję okna RANK() i pozwala znaleźć sprzedawców o najwyższym przychodzie. Dodatkowo pokazuje liczbę zamówień, region i średnią ocenę.

Fragment raportu sprzedawców z CTE i funkcją RANK():

```
WITH seller_sales AS (
    SELECT
        ds.seller_id,
        COUNT(DISTINCT foi.order_id) AS total_orders,
        SUM(ISNULL(foi.total_item_value, 0) + ISNULL(foi.freight_value, 0)) AS total_revenue,
        AVG(CAST(fr.review_score AS DECIMAL(5, 2))) AS avg_review_score
    FROM fact_order_items foi
    JOIN dim_seller ds ON foi.seller_key = ds.seller_key
    LEFT JOIN fact_reviews fr ON foi.order_id = fr.order_id
    GROUP BY ds.seller_id
)
SELECT TOP 20
    RANK() OVER (ORDER BY total_revenue DESC) AS revenue_rank,
    seller_id,
    total_orders,
    total_revenue,
    avg_review_score
FROM seller_sales;
```

12. Podstawowa obsługa delty danych

Podstawowa delta danych została zaprezentowana w pliku 5_delta_demo.sql. Skrypt dopisuje tylko takie rekordy, których jeszcze nie ma w hurtowni. Wykorzystano warunek NOT EXISTS dla fact_orders oraz fact_order_items. Po wykonaniu delty odświeżana jest tabela agregująca agg_monthly_sales.

Najważniejszy mechanizm delty danych:

```
WHERE NOT EXISTS (
    SELECT 1
    FROM fact_orders fo
    WHERE fo.order_id = o.order_id
);
```

```
WHERE NOT EXISTS (
    SELECT 1
    FROM fact_order_items foi
    WHERE foi.order_id = oi.order_id
      AND foi.order_item_id = oi.order_item_id
);
```

```
11:06:18 PM    Started executing query at  Line 1
               Commands completed successfully.
11:06:18 PM    Started executing query at  Line 3
               (0 rows affected)
11:06:19 PM    Started executing query at  Line 60
               (0 rows affected)
11:06:19 PM    Started executing query at  Line 91
               (24902 rows affected)
               Warning: Null value is eliminated by an aggregate or other SET operation.
               (24902 rows affected)
               Total execution time: 00:00:01.852
```

Rysunek 8. Wynik uruchomienia demonstracji delty danych.

Wynik 0 rows affected oznacza, że po pełnym wcześniejszym ładowaniu nie znaleziono nowych rekordów do dopisania. Następnie odświeżono agregację miesięczną, co potwierdzają komunikaty 24902 rows affected. Ostrzeżenie o NULL pojawia się przy agregacji i nie zatrzymuje wykonania skryptu.

13. Logowanie ETL i walidacje audytowe

Jako element dodatkowy dodano tabele audytowe etl_run_log oraz etl_validation_result. Pozwalają one zapisać wynik ręcznej walidacji po wykonaniu ETL: status, liczbę rekordów w tabelach faktów i agregacji oraz wyniki kontroli jakości danych.

Tabela logowania uruchomień ETL:

```
CREATE TABLE dbo.etl_run_log (
    etl_run_id INT IDENTITY(1,1) PRIMARY KEY,
    run_name VARCHAR(100) NOT NULL,
    started_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    finished_at DATETIME2 NULL,
    status VARCHAR(30) NOT NULL,
    fact_orders_rows INT NULL,
    fact_order_items_rows INT NULL,
    fact_payments_rows INT NULL,
    fact_reviews_rows INT NULL,
    agg_monthly_sales_rows INT NULL,
    error_message VARCHAR(1000) NULL
);
```

	etl_run_id	run_name	started_at	finished_at	status	fact_orders_rows	fact_order_items_rows	fact_payments_rows	fact_reviews_rows	agg_monthly_sales_rows	error_message
1	2	manual_validation_after_etl	2026-05-07 21:07:33.4167801	2026-05-07 21:07:33.4714639	SUCCESS	99441	112650	103886	97685	24902	NULL
2	1	manual_validation_after_etl	2026-05-07 15:19:29.2859542	2026-05-07 15:19:29.3820592	SUCCESS	99441	112650	103886	97685	24902	NULL

	validation_result_id	etl_run_id	validation_name	validation_status	expected_value	actual_value	details	checked_at
1	7	2	fact_orders is not empty	OK	> 0	99441	Tabela fact_orders powinna zawierac zaladowane zam…	2026-05-07 21:07:33.4335238
2	8	2	fact_order_items is not empty	OK	> 0	112650	Tabela fact_order_items powinna zawierac pozycje z…	2026-05-07 21:07:33.4335238
3	9	2	agg_monthly_sales is not empty	OK	> 0	24902	Tabela agregujaca powinna byc zasilona po ETL.	2026-05-07 21:07:33.4335238
4	10	2	missing customer keys in fact_orders	OK	0	0	Sprawdzenie, czy fact_orders ma poprawne klucze kl…	2026-05-07 21:07:33.4335238
5	11	2	missing product keys in fact_order_items	OK	0	0	Sprawdzenie, czy fact_order_items ma poprawne kluc…	2026-05-07 21:07:33.4335238
6	12	2	duplicate product business keys	OK	0	0	Sprawdzenie duplikatów product_id w dim_product.	2026-05-07 21:07:33.4335238

Rysunek 9. Wyniki logowania ETL i walidacji audytowych.

W tabeli etl_run_log widoczne są dwa uruchomienia walidacji zakończone statusem SUCCESS. W tabeli etl_validation_result wszystkie pokazane walidacje mają status OK, m.in. niepuste fact_orders, fact_order_items, agg_monthly_sales oraz brakujące klucze równe 0.

14. Slowly Changing Dimensions - SCD Type 2

W podstawowym modelu wymiary działają jak SCD Type 1, czyli przechowują aktualny stan. Dodatkowo przygotowano demonstracyjny wymiar dim_product_scd2, który pokazuje mechanizm SCD Type 2. Zmiana atrybutu produktu zamyka poprzednią wersję przez ustawienie valid_to i is_current = 0 oraz tworzy nową aktualną wersję z is_current = 1.

Mechanizm zamknięcia starej wersji i utworzenia nowej wersji SCD2:

```
UPDATE target_product
SET
    valid_to = DATEADD(SECOND, -1, @change_time),
    is_current = 0,
    updated_at = @change_time
FROM dbo.dim_product_scd2 target_product
JOIN #changed_product source_product
    ON target_product.product_id = source_product.product_id
WHERE target_product.is_current = 1
  AND target_product.scd_hash <> source_product.new_hash;

INSERT INTO dbo.dim_product_scd2 (..., valid_from, valid_to, is_current, scd_hash)
SELECT ..., @change_time, NULL, 1, new_hash
FROM #changed_product;
```

	product_id	category_name_pt	category_name_en	valid_from	valid_to	is_current
1	00066f42aeeb9f3007548bb9d3f33c38	perfumaria	perfumery	1900-01-01 00:00:00.0000000	2026-05-07 15:21:58.0900487	0
2	00066f42aeeb9f3007548bb9d3f33c38	perfumaria_SCD_DEMO	perfumery SCD Demo	2026-05-07 15:21:59.0900487	NULL	1
3	00088930e925c41fd95ebfe695fd2655	automotivo	auto	1900-01-01 00:00:00.0000000	2026-05-07 21:10:27.8757573	0
4	00088930e925c41fd95ebfe695fd2655	automotivo_SCD_DEMO	auto SCD Demo	2026-05-07 21:10:28.8757573	NULL	1

	all_scd2_rows	distinct_products	current_rows	historical_rows
1	32953	32951	32951	2

Rysunek 10. Demonstracja SCD Type 2 dla produktów.

W wyniku widoczne są produkty z wersją historyczną i aktualną. Podsumowanie pokazuje 32953 rekordy SCD2, 32951 produktów, 32951 rekordów aktualnych i 2 rekordy historyczne.

15. Integracja z API NBP i przeliczenie BRL na PLN

Projekt został rozszerzony o integrację z zewnętrznym źródłem danych. Skrypt fetch_nbp_brl_rate.py pobiera aktualny kurs BRL/PLN z API NBP i zapisuje go w tabeli dim_exchange_rate. Następnie widok vw_fact_orders_pln przelicza wartość zamówień z BRL na PLN.

Fragment pobrania kursu z API NBP:

```
NBP_URL = "https://api.nbp.pl/api/exchangerates/rates/a/brl/?format=json"

response = requests.get(NBP_URL, headers={"Accept": "application/json"}, timeout=30)
payload = response.json()
rate = payload["rates"][0]
```

Widok przeliczający zamówienia z BRL na PLN:

```
CREATE OR ALTER VIEW dbo.vw_fact_orders_pln AS
SELECT
    fo.order_id,
    fo.total_order_value AS total_order_value_brl,
    er.rate_value AS brl_to_pln_rate,
    CAST(fo.total_order_value * er.rate_value AS DECIMAL(18,2)) AS total_order_value_pln,
```

```
    er.rate_date AS exchange_rate_date
FROM dbo.fact_orders fo
CROSS APPLY (
    SELECT TOP 1 rate_value, rate_date
    FROM dbo.dim_exchange_rate
    WHERE source_system = 'NBP'
      AND currency_code = 'BRL'
      AND base_currency = 'PLN'
    ORDER BY rate_date DESC, fetched_at DESC
) er;
```

	order_id	customer_key	order_date_key	total_order_value_brl	brl_to_pln_rate	total_order_value_pln	exchange_rate_date
1	00010242fe8c5a6d1ba2dd792cb16214	23675	20170913	72.19	0.729700	52.68	2026-05-07
2	00018f77f2f0320c557190d7a144bdd3	95963	20170426	259.83	0.729700	189.60	2026-05-07
3	000229ec398224ef6ca0657da4fc703e	38903	20180114	216.87	0.729700	158.25	2026-05-07
4	00024acbcdf0a6daa1e931b038114c75	82958	20180808	25.78	0.729700	18.81	2026-05-07
5	00042b26cf59d7ce69dfabb4e55b4fd9	34382	20170204	218.04	0.729700	159.10	2026-05-07
6	00048cc3ae777c65dbb7d2a0634bc1ea	50184	20170515	34.59	0.729700	25.24	2026-05-07
7	00054e8431b9d7675808bcb819fb4a32	19737	20171210	31.75	0.729700	23.17	2026-05-07
8	000576fe39319847cbb9d288c5617fa6	61707	20180704	880.75	0.729700	642.68	2026-05-07
9	0005a1a1728c9d785b8e2b08b904576c	8540	20180319	157.60	0.729700	115.00	2026-05-07
10	0005f50442cb953dcd1d21e1fb923495	20619	20180702	65.39	0.729700	47.72	2026-05-07

Rysunek 11. Widok vw_fact_orders_pln z wartościami zamówień przeliczonymi na PLN.

Na screenie widać kurs 0.729700 oraz datę kursu 2026-05-07. Przykładowe zamówienie o wartości 72.19 BRL zostało przeliczone na 52.68 PLN.

16. Raporty zaawansowane SQL

W pliku 9_advanced_reports.sql przygotowano raporty wykorzystujące CTE, funkcje okna i bardziej złożone agregacje. Raporty obejmują dynamikę miesięczną sprzedaży, ranking kategorii z udziałem w przychodzie, segmentację klientów, wpływ opóźnień na oceny oraz zamówienia przeliczone na PLN.

Przykłady technik SQL użytych w raportach zaawansowanych:

```
LAG(monthly_revenue) OVER (ORDER BY year, month_of_year) AS previous_month_revenue

RANK() OVER (ORDER BY total_revenue DESC) AS category_rank

NTILE(3) OVER (ORDER BY SUM(total_order_value) DESC) AS customer_segment
```

	year	month_of_year	total_orders	monthly_revenue	previous_month_revenue	revenue_mom_percent
1	2016	9	4	354.75	NULL	NULL
2	2016	10	324	56808.84	354.75	15913.77
3	2016	12	1	19.62	56808.84	-99.97
4	2017	1	800	137188.49	19.62	699127.78
5	2017	2	1780	286280.62	137188.49	108.68
6	2017	3	2682	432048.59	286280.62	50.92
7	2017	4	2404	412422.24	432048.59	-4.54
8	2017	5	3700	586190.95	412422.24	42.13
9	2017	6	3245	502963.04	586190.95	-14.20

	category_rank	category_name_en	total_orders	total_revenue	revenue_share_percent	cumulative_revenue_share_percent
1	1	health_beauty	8836	1441248.07	9.10	9.10
2	2	watches_gifts	5624	1305541.61	8.24	17.34
3	3	bed_bath_table	9417	1241681.72	7.84	25.17
4	4	sports_leisure	7720	1156656.48	7.30	32.47
5	5	computers_accessories	6689	1059272.40	6.69	39.16
6	6	furniture_decor	6449	902511.79	5.70	44.86
7	7	housewares	5884	778397.77	4.91	49.77
8	8	cool_stuff	3632	719329.95	4.54	54.31
9	9	auto	3897	685384.32	4.33	58.64

	customer_segment	region_name	customers_count	avg_orders_count	avg_total_spent	segment_revenue
1	High value	Southeast	15729	1.00	368.99	5803853.52
2	Medium value	Southeast	16713	1.00	136.36	2278914.10
3	Low value	Southeast	35824	1.00	60.00	2149555.88
4	High value	South	3597	1.00	380.40	1368302.02
5	High value	Northeast	3177	1.00	405.55	1288442.80
6	High value	Center-West	1624	1.00	400.81	650911.36
7	Medium value	South	3684	1.00	136.26	501965.82
8	Low value	South	6867	1.00	62.14	426707.96
9	Medium value	Northeast	2527	1.00	137.31	346982.15

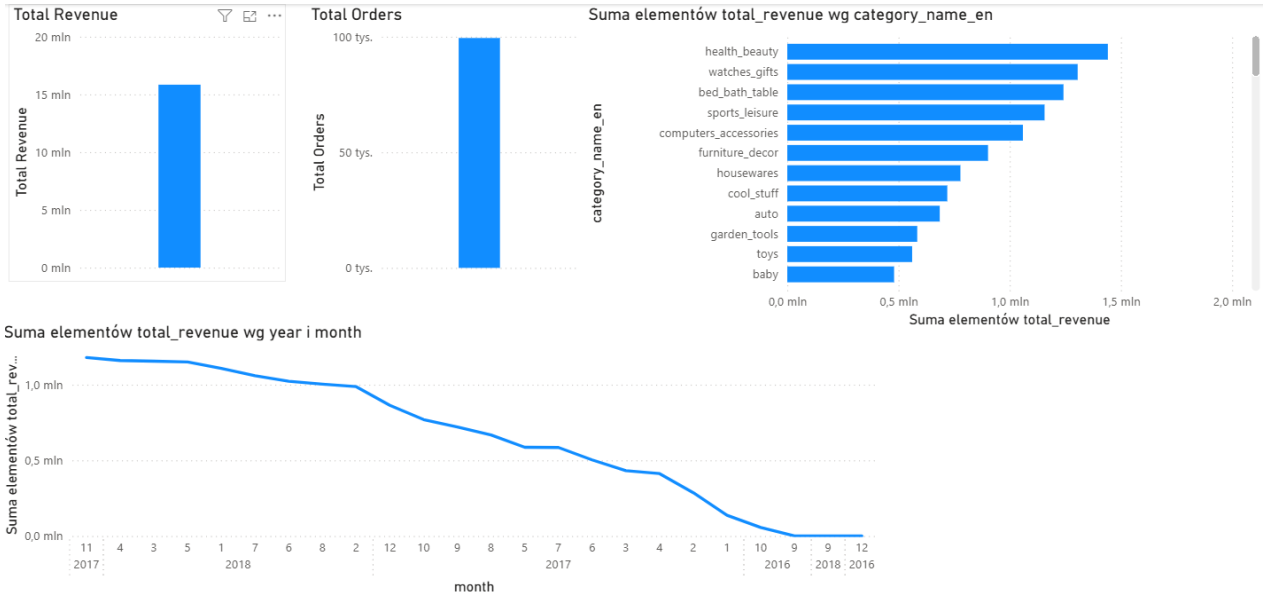
	delivery_status	orders_count	avg_days_to_delivery	avg_review_score	total_revenue
1	Delayed	7827	31.48	2.57	1353901.23
2	On time / no delay	91614	10.83	4.24	14522542.04

Rysunek 12. Raporty zaawansowane SQL: dynamika sprzedaży, ranking kategorii, segmentacja klientów i wpływ opóźnień.

Raport zaawansowany pokazuje m.in., że najwyższy udział w przychodzie ma kategoria health_beauty z przychodem 1 441 248.07 i udziałem 9.10%. Dodatkowo widać istotną różnicę w ocenach między zamówieniami opóźnionymi i terminowymi: zamówienia opóźnione mają średnią ocenę 2.57, a zamówienia terminowe lub bez opóźnienia 4.24.

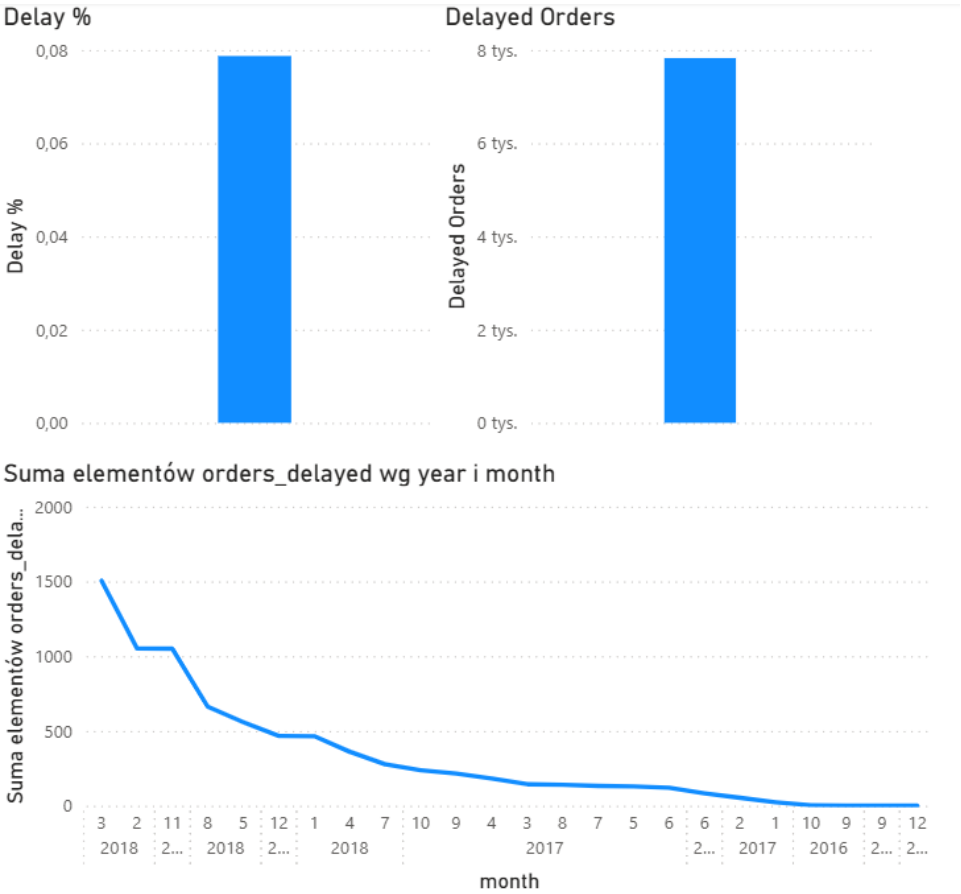
17. Raportowanie w Power BI

Oprócz raportów SQL przygotowano dashboardy w Power BI. Dane zostały zaimportowane z bazy olist_dwh. Raport obejmuje trzy główne widoki: Sprzedaż, Dostawy oraz Sprzedawcy/Kategorie.



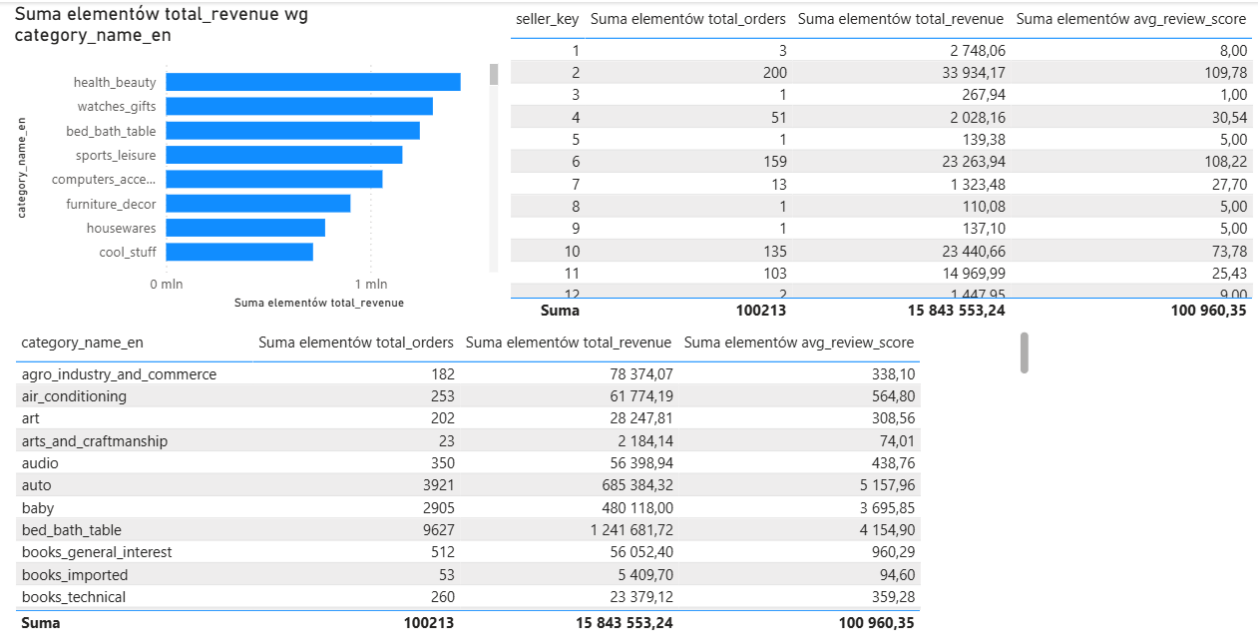
Rysunek 13. Power BI - strona Sprzedaż.

Strona Sprzedaż przedstawia całkowity przychód, liczbę zamówień, przychód według kategorii oraz trend miesięczny. Największe kategorie widoczne na dashboardzie to m.in. health_beauty, watches_gifts, bed_bath_table i sports_leisure.



Rysunek 14. Power BI - strona Dostawy.

Strona Dostawy pokazuje odsetek i liczbę opóźnionych zamówień oraz trend opóźnień w czasie. Wizualizacja ułatwia szybkie wskazanie okresów z większą liczbą opóźnień.



seller_key	Suma elementów total_orders	Suma elementów total_revenue	Suma elementów avg_review_score
1	3	2 748,06	8,00
2	200	33 934,17	109,78
3	1	267,94	1,00
4	51	2 028,16	30,54
5	1	139,38	5,00
6	159	23 263,94	108,22
7	13	1 323,48	27,70
8	1	110,08	5,00
9	1	137,10	5,00
10	135	23 440,66	73,78
11	103	14 969,99	25,43
12	2	1 447,95	9,00
Suma	**100213**	**15 843 553,24**	**100 960,35**

category_name_en	Suma elementów total_orders	Suma elementów total_revenue	Suma elementów avg_review_score
agro_industry_and_commerce	182	78 374,07	338,10
air_conditioning	253	61 774,19	564,80
art	202	28 247,81	308,56
arts_and_craftmanship	23	2 184,14	74,01
audio	350	56 398,94	438,76
auto	3921	685 384,32	5 157,96
baby	2905	480 118,00	3 695,85
bed_bath_table	9627	1 241 681,72	4 154,90
books_general_interest	512	56 052,40	960,29
books_imported	53	5 409,70	94,60
books_technical	260	23 379,12	359,28
Suma	**100213**	**15 843 553,24**	**100 960,35**

Rysunek 15. Power BI - strona Sprzedawcy i kategorie.

Strona Sprzedawcy/Kategorie porównuje kategorie produktów i sprzedawców pod względem przychodu, liczby zamówień i ocen. Tabela oraz wykres słupkowy umożliwiają szybkie znalezienie najbardziej dochodowych kategorii.

18. Wnioski

Projekt spełnia główne wymagania zaliczeniowe: posiada działający model hurtowni danych, więcej niż jedną tabelę faktów, wiele wymiarów z wymiarem dat, proces ETL, transformacje, agregację miesięczną, podstawową deltę danych, raporty SQL oraz dokumentację uruchomienia. Dodatkowo projekt zawiera walidacje, logowanie ETL, demonstrację SCD Type 2, integrację z API NBP i dashboardy Power BI.

Analiza danych wykazała, że największy wolumen i przychód koncentruje się w regionie Southeast, natomiast najwyższy procent opóźnionych dostaw wystąpił w regionie Northeast. Raporty zaawansowane potwierdzają również negatywny wpływ opóźnień na oceny klientów - zamówienia opóźnione mają znacznie niższą średnią ocenę niż zamówienia dostarczone na czas lub bez opóźnienia.

Najważniejszą korzyścią z zastosowania hurtowni danych jest oddzielenie surowych danych źródłowych od modelu analitycznego. Dzięki temu raporty mogą korzystać z uporządkowanych wymiarów, faktów i agregacji, a użytkownik biznesowy nie musi analizować bezpośrednio wielu surowych plików CSV.

19. Zawartość paczki do oddania

Element	Czy dołączyć	Uwagi
Kod Python	Tak	main_staging.py, main_dwh.py, fetch_nbp_brl_rate.py, scripts/
Skrypty SQL	Tak	1_staging_tables.sql - 9_advanced_reports.sql
docker-compose.yml	Tak	do uruchomienia SQL Servera
requirements.txt	Tak	zależności Pythona
Dokumentacja	Tak	plik PDF lub DOCX/MD według wymagań prowadzącego
Power BI	Opcjonalnie/Tak	plik .pbix jako element dodatkowy
Screeny	Tak	mogą być w folderze screenshots lub osadzone w PDF
Pliki CSV	Zależnie od rozmiaru	jeżeli są duże, wystarczy opis i wskazanie źródła danych

.venv, __pycache__	Nie	nie oddawać plików tymczasowych i środowiska lokalnego

20. Szybka lista komend kontrolnych

```
# Start projektu po ponownym uruchomieniu komputera
docker compose up -d
.\.venv\Scripts\activate

# Kontrola kontenera
docker ps

# Pełne odtworzenie stagingu i DWH
python main_staging.py --force
python main_dwh.py --etl --force

# Pobranie kursu NBP
python fetch_nbp_brl_rate.py

# Pliki SQL do uruchomienia w VS Code / mssql
4_validation_queries.sql
5_delta_demo.sql
6_etl_audit_schema.sql
7_scd_type2_product.sql
8_external_nbp_rates.sql
9_advanced_reports.sql
```